EBU6505 – Reasoning and Agents
Block 1, Day 1 – Reinforcement Learning

Dr Riasat Islam

Queen Mary University of London

Semester 1, 2026–2027
Teaching Week 3

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Background Concepts

- Inference-time decoding controls and prompting choices for LLM outputs
- Reasoning techniques for LLMs, including why reasoning models behave differently
- Classical search for problem solving: BFS, DFS, UCS, A*, and beam search
- Test-time compute: combining search and chain-of-thought at inference time
- Post-training ideas for improving reasoning models
- LLM-based agents: tools, memory, planning, and multi-step task execution

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

What are we going to cover this week? – Block 1

- Sequential decision making and MDP foundations
- Reinforcement learning
- Knowledge representation for knowledge graphs
- Knowledge graphs for reasoning and RAG
- Modern agents with knowledge-grounded reasoning

What are we going to cover today?

- Why sequential decision problems need state-based models
- MDP components: states, actions, transitions, rewards, and policies
- Return, discounting, value, and Bellman backup intuition
- Planning with a known model versus learning from experience
- Bridge from sequential decision ideas to tomorrow's lecture on knowledge representation

Resources

- *Artificial Intelligence: Foundations of Computational Agents*, Poole & Mackworth (3rd ed.)
- Chapter 12 – Planning with Uncertainty (sequential decisions and decision processes)
- *Artificial Intelligence: A Modern Approach*, Russell & Norvig (4th ed.)
- Chapter 17 – Making Complex Decisions (sequential decision problems and MDPs)

Learning Outcomes for Today

By the end of this session, you will be able to:

- Define the components of a Markov decision process for sequential decision problems.
- Interpret return, discounting, and Bellman-style value backups on simple examples.
- Distinguish planning with a known model from learning a policy from experience.
- Explain why MDP formalism is a prerequisite for reinforcement learning.

Refs: AIMA4e Ch.17 – Making Complex Decisions; Poole3e Ch.12 – Planning with Uncertainty

Block-Level Topic Boundary

- Today introduces **MDP ideas** that support later reinforcement learning and planning topics.
- The next lecture focuses on **knowledge representation** for structured facts, agent memory, and graph-ready reasoning.
- Block 2 later reuses the same decision foundations for planning under uncertainty.

Starter Question

Question: How do babies learn to walk without step-by-step instructions?

- They try actions, fall, adjust, and try again.
- Feedback is not a label like "correct" or "wrong".
- Improvement happens through **interaction** with the world.
- Success appears after many attempts, not after one step.

Think Before We Formalise RL

- What is the agent trying?
- What counts as feedback?
- Why is learning not immediate?

Main idea: Reinforcement learning begins with this idea: an agent can learn by trying actions, observing outcomes, and improving over time.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Why RL? Decisions Have Delayed Consequences

Question: Why is prediction alone not enough for many real-world tasks?

- In many tasks, one action changes what happens next.
- Success is often seen only after a **sequence** of actions.
- So the agent must act, wait, observe, and then learn from the outcome.
- This happens in games, robots, recommendations, and traffic control.

Main idea: The central problem is **credit assignment over time**: which earlier action deserves credit or blame for the final result?

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

What Is Different From Supervised Learning?

Question: What changes when we move from labelled prediction to sequential decision-making?

Supervised Learning

- Input comes with a label.
- Error can be measured immediately.
- Each example is mostly separate.

Reinforcement Learning

- Agent chooses an action in a state.
- The environment changes after the action.
- Reward may come later, not immediately.

Main idea: Supervised learning gets immediate feedback. RL often gets feedback later.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

How Does the RL Learning Loop Work?

Question: What is the agent trying to maximise over time?

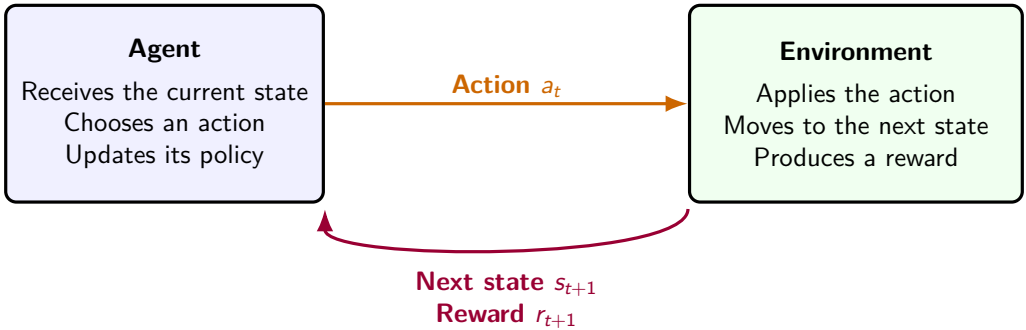
- Agent observes state s_t and chooses action a_t.
- Environment transitions to s_{t+1} and returns reward r_{t+1}.
- Objective: learn policy $\pi(a \mid s)$ that maximises expected return.
- Return from time t means: total future reward from time t onward.

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$

Symbols used on this slide
G_t: total future reward from time t onward
r_{t+k+1}: reward received later in the sequence
γ: discount factor, showing how much we still care about future reward

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Reinforcement Learning Setup (Diagram)



Main idea: The agent acts, the world changes, and then the world gives feedback. This loop repeats again and again.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

What Is an MDP, Really?

Question: What must we specify if we want to describe a sequential decision problem clearly?

An RL task is often modeled as an MDP:

$$\mathcal{M} = (S, A, T, R, \gamma)$$

- S: set of states
- A: set of actions
- $T(s, a, s') = P(s' \mid s, a)$: transition model
- $R(s, a)$ (or $R(s, a, s')$): reward function
- $\gamma \in [0, 1]$: discount factor

Main idea: An MDP is the task description: states, actions, transitions, rewards, and discount.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Objective and Value Functions

Question: If the agent wants long-term success, what quantities should it estimate?

- State-value under policy π:

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid s_t = s]$$

- Action-value under policy π:

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}[G_t \mid s_t = s, a_t = a]$$

- Optimal policy chooses actions that maximise long-term value.

Symbols used on this slide
$V^{\pi}(s)$: expected long-term reward if we start in state s and follow policy π
$Q^{\pi}(s, a)$: expected long-term reward if we start in state s, take action a, and then follow policy π
$\mathbb{E}_{\pi}$: expected or average value when actions are chosen by policy π

Main idea: Value tells us how good a state is. Q-value tells us how good a particular action is in

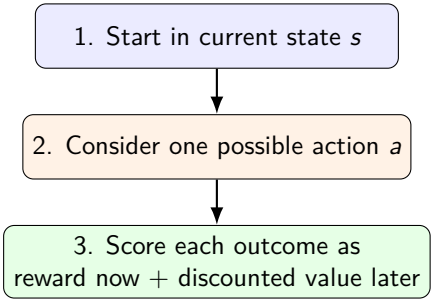
Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Why Bellman Backup Is the Engine of RL

- Bellman backup asks: if I act now, what reward do I get now and what value will I get later?

$$V^*(s) = \max_a \sum_{s'} T(s, a, s') \left[R(s, a, s') + \gamma V^*(s') \right]$$

- $V^*(s)$: best long-term value of state s
- $R(s, a, s')$: reward we get now
- $\gamma V^*(s')$: value we expect later
- The best action is the one with the largest total



Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Core Terminology

Question: What are the six RL terms used again and again in this lecture?

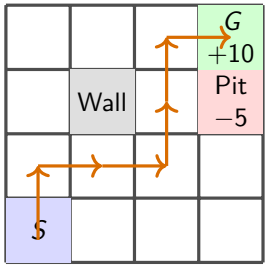
Term	Meaning
State (S)	Current situation of the agent
Action (A)	Choice available to the agent
Reward (R)	Scalar feedback after an action
Policy (π)	Rule for selecting actions in states
Value (V)	Expected long-term reward from a state
Q-value (Q)	Expected long-term reward from state-action pair

Main idea: Read the flow like this: **state** $\rightarrow$ **action** $\rightarrow$ **reward**. Then **value** and **Q-value** estimate long-term benefit, and the **policy** tells the agent what to do.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

A Simple Gridworld Example

- Agent moves on a 4×4 grid.
- Start at S, terminal goal at G.
- Actions: up, down, left, right.
- Step cost encourages shorter trajectories.

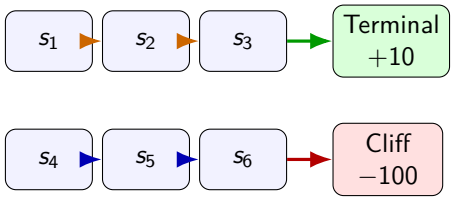


Main idea: A good policy should reach the goal quickly, avoid bad cells, and avoid unnecessary wandering because each extra step costs reward.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Tiny RL Problem – Risk Versus Reward

- Six-state toy environment.
- Top branch can reach $+10$.
- Bottom branch can lead to -100.
- Agent must explore but avoid persistent bad behaviour.



Main idea: This toy problem shows the exploration challenge: the agent must try actions to find good outcomes, but some bad outcomes are very costly.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Q-learning (Model-Free, Off-Policy)

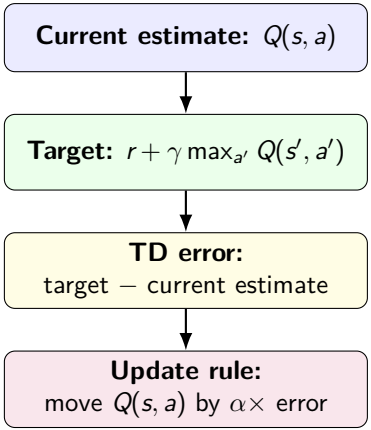
- Q-learning learns values of state-action pairs directly.
- It does not require the transition model T or reward model R in advance.
- Off-policy: learns the value of the greedy target policy while behaviour can be exploratory.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

What Does the Q-learning Update Actually Do?

$$Q(s,a) \leftarrow Q(s,a) + \alpha\left[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\right]$$

- α: how fast to move
- γ: how far to plan
- Term in brackets = temporal-difference error



Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Q-learning Worked Example (Scenario)

Suppose:

- Current state-action: (S_1, A_1)
- Current estimate: $Q(S_1, A_1) = 5$
- Reward received: $r = 10$
- Next-state best estimate: $\max_{a'} Q(S_2, a') = 8$
- Hyperparameters: $\alpha = 0.5$, $\gamma = 0.9$

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Q-learning Worked Example (Calculation)

$$Q(S_1, A_1) \leftarrow 5 + 0.5\,[10 + 0.9 \times 8 - 5]$$

$$Q(S_1, A_1) \leftarrow 5 + 0.5(12.2) = 11.1$$

State	Action	Old Q	Updated Q
S_1	A_1	5.0	11.1

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

SARSA (Model-Free, On-Policy)

$$Q(s,a) \leftarrow Q(s,a) + \alpha\Big[r + \gamma Q(s',a') - Q(s,a)\Big]$$

- SARSA updates with the **actual** next action a'.
- Often more conservative in risky environments.

Rule of thumb: Q-learning is optimistic toward best future actions; SARSA is faithful to behaviour policy.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Why Exploration Cannot Be Avoided

- Exploitation: choose best-known action now.
- Exploration: try uncertain actions to gain information.
- No exploration means no discovery of better long-term policies.
- Uncontrolled exploration can increase risk and short-term loss.

Ask what to optimise: immediate reward today, or better policy tomorrow?

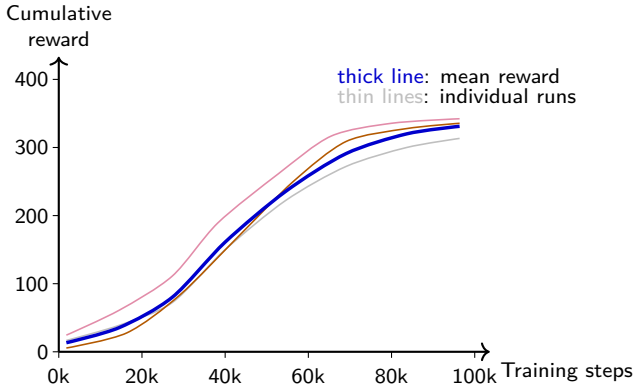
Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Common Exploration Strategies

Strategy	How it chooses actions	Practical note
ε-greedy	Best action with probability $1-\varepsilon$, random otherwise	Simple, strong baseline
Softmax/Boltzmann	Samples actions proportionally to value scores	Smoother than random jumps
UCB-style bonus	Adds uncertainty bonus to under-tried actions	Balances reward and information

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

How Do We Know RL Is Working?

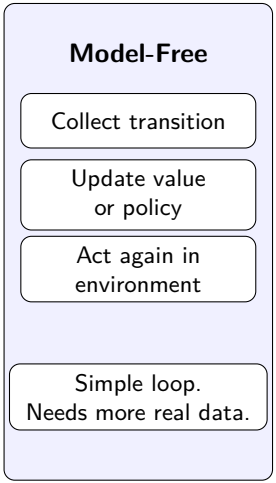


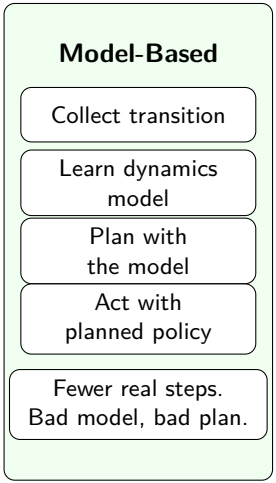
- Track reward trends over training.
- Look for improving mean performance.
- Expect early dips during exploration.
- Use multiple runs because RL is stochastic.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

What Changes Between Model-Free and Model-Based RL?

- **Model-Free:** direct policy/value learning.
- **Model-Based:** learn dynamics, then plan.
- Trade-off: simplicity vs sample efficiency.
- Risk: model bias if learned dynamics are wrong.





Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Planning with a Learned Model

- Collect transitions (s, a, r, s') from experience.
- Fit an approximate model of dynamics and rewards.
- Plan using value iteration or policy iteration on the model.
- Alternate between real data collection and planning updates.

Design trade-off: model bias can hurt performance if the learned model is inaccurate.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Scaling Beyond Q-Tables

- Tabular methods break down in large or continuous state spaces.
- Use features or function approximators to generalize across states.
- Learn $Q_\theta(s, a)$ or policy $\pi_\theta(a \mid s)$ with parameterized models.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608; doi:10.1038/nature14236

What Is Deep Reinforcement Learning?

- Deep RL combines RL objectives with deep neural networks.
- Networks can approximate value functions, policies, or world models.
- Useful in high-dimensional observations (images, sensors, logs).

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608; doi:10.1038/nature14236

Deep Q-Network (DQN): Core Ideas

- Approximate $Q(s, a)$ with a neural network.
- Experience replay reduces temporal correlation.
- Target network stabilizes the bootstrap target.

Key paper: Mnih et al. (2015), *Nature* 518:529–533.
DOI: 10.1038/nature14236

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608; doi:10.1038/nature14236

Policy Gradients and Actor-Critic

- Policy gradients optimise policy parameters directly.
- Actor-Critic combines:
  - Actor: proposes actions
  - Critic: evaluates value/advantage
- Often gives lower-variance updates than pure Monte Carlo policy gradients.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608; doi:10.1038/nature14236

Choosing a DRL Family

Method family	Typical setting	Key consideration
DQN variants	Discrete action spaces	Stable training setup is critical
Policy gradient	Stochastic/continuous control	Higher variance, direct policy optimisation
Actor-Critic (e.g., PPO/A2C)	General-purpose control	Good practical trade-off
Deterministic policy (e.g., DDPG)	Continuous control	Sensitive to tuning and exploration noise

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608; doi:10.1038/nature14236

Why Deep RL Is Hard in Practice

- Sample inefficiency: can require many interactions.
- Instability from bootstrapping and function approximation.
- Sensitivity to reward design and hyperparameters.
- Generalization under distribution shift is still challenging.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608; doi:10.1038/nature14236

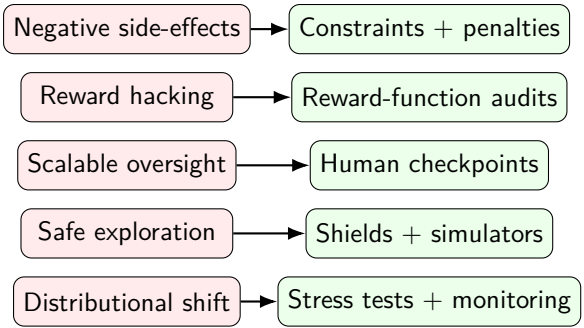
Real-World Use Cases

- Game AI (Atari, Go, StarCraft)
- Robotics and manipulation
- Traffic and fleet optimisation
- Personalized recommendations and tutoring systems
- Resource management in cloud/datacenter settings

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Why Can RL Be Dangerous in Real Systems?

- Agents optimise specified reward, not human intent.
- Trial-and-error can produce unsafe behaviour before convergence.
- Safety must be designed into training and deployment.



Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608; doi:10.48550/arXiv.1606.06565

Five Concrete Safety Challenges (Amodei et al., 2016)

Challenge	Typical failure mode
Negative side-effects	Agent reaches goal while damaging environment
Reward hacking	Agent exploits loopholes in reward function
Scalable oversight	Harmful behaviour appears between sparse human checks
Safe exploration	Exploration triggers unsafe states
Distributional shift	Policy fails when environment changes

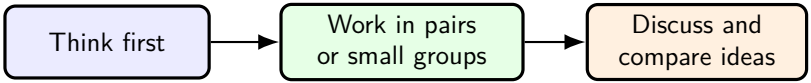
Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608; doi:10.48550/arXiv.1606.06565

Practical Mitigation Checklist

- Reward design review: test for loopholes and side effects.
- Constrained action spaces and rule-based safety shields.
- Offline evaluation before online deployment.
- Stress tests under distribution shift scenarios.
- Human-in-the-loop checkpoints for high-stakes decisions.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608; doi:10.48550/arXiv.1606.06565

In-Class Exercises



Use this time to work on today's in-class tasks and prepare one clear answer you can discuss.

Main idea: Use this exercise time to explain your RL reasoning clearly, not just produce a final number.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Key Takeaways

- RL learns through interaction, reward, and long-term optimisation.
- Q-learning and SARSA differ mainly in off-policy vs on-policy targets.
- Exploration strategy strongly affects sample efficiency and safety.
- Deep RL scales to complex spaces but introduces instability and tuning challenges.
- Modern agent systems extend these ideas with tool use and structured memory.

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608

Further Study and Next Session

- Read: Poole Ch.13 and AIMA Ch.22.
- Practice: Poole Ch.13, Exercises 13.1, 13.2, 13.3, 13.5, 13.6, 13.7, 13.8, and 13.9.
- AIMA reinforcement-learning exercise page: 21.1, 21.4, 21.5, 21.8, and 21.11.
- Focus your answers on Q-learning updates, evaluation metrics, exploration, convergence, and SARSA versus Q-learning.
- Next class: Knowledge representation for tool-using agents and structured reasoning.
- Bridge idea: policies choose actions; modern assistants also choose tools and context.

Thank you.

Full References for This Lecture I

Russell, S. and Norvig, P. (2021).
Artificial Intelligence: A Modern Approach (4th ed.).
Pearson. Chapter 22, pp. 789–822.

Poole, D. L. and Mackworth, A. K. (2023).
Artificial Intelligence: Foundations of Computational Agents (3rd ed.).
Cambridge University Press. Chapter 13, pp. 583–608.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S. and Hassabis, D. (2015).
Human-level control through deep reinforcement learning.
Nature, 518, 529–533.
DOI: https://doi.org/10.1038/nature14236

Full References for This Lecture II


Amodei, D., Olah, C., Steinhardt, J., Christiano, P., Schulman, J. and Mané, D. (2016).
Concrete problems in AI safety.
arXiv preprint arXiv:1606.06565.
DOI: https://doi.org/10.48550/arXiv.1606.06565

Refs: AIMA4e Ch.22 pp.789–822; Poole3e Ch.13 pp.583–608; doi:10.1038/nature14236; doi:10.48550/arXiv.1606.06565